EVOLUTIONARY ALGORITHM

LOUIS DAVID – LUCAS NEANDER

GOALS

Generate soft robots using the EvoGym suite

- Walker-v0 (easy)
- Thrower-v0 (medium)
- Climb-v2 (hard)

Constrained with:

- a budget of 5 000 000 steps
- Our choice of EA
- Gene representaiton
- Evolution of hyperparameters

STRATEGY

Algorithm 1 Algorithmic framework of robot evolution

Inputs:

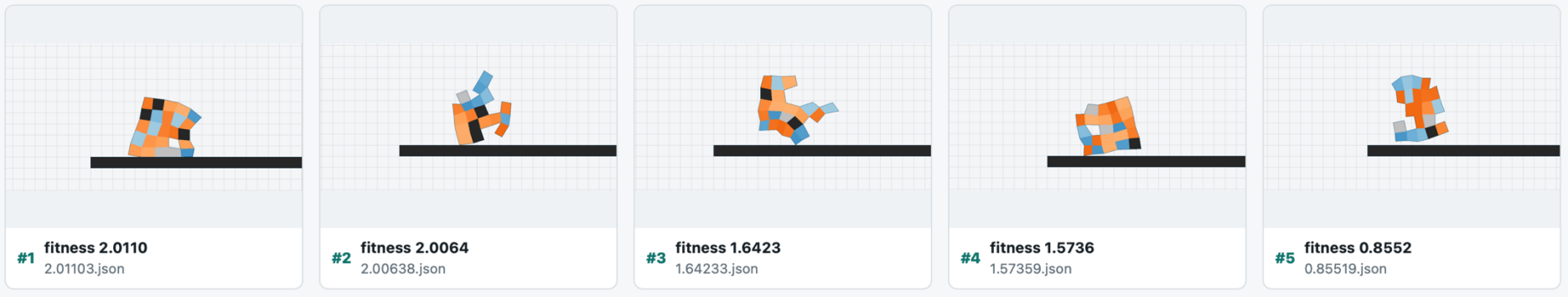
n: Number of generations
$N_1, \ldots, N_n$: Population sizes per generation
$P_1, \ldots, P_n$: Number of parents per generation
$p_1, \ldots, p_n$: Body mutation probabilities per generation
$r_1, \ldots, r_n$: Number of generations per optimization
$q_1, \ldots, q_n$: Sub-Population sizes per optimization
$m_1, \ldots, m_n$: Max steps per optimization

Outputs: D^* best robot design and C^* best controller design

```
S ← RandomRobots(N_1)                          // Initial population with random robot designs
for i ← 1 to n do
    for j ← 1 to N_i do
        C_j ← OptimizeRobot(r_i, q_i, m_i)     // Optimize the controller of given robot design
        f_j ← EvaluateRobot(S_j, C_j)          // Evaluate the fitness of given design and controller
    end for
    S ← BestSelection(S, C, f, P_i)            // Select the best performing designs for the generation
    S ← Mutate(S, p_i)                         // Generate new population by mutating the body of the best designs
end for
Find the best design D* and controller C* with maximum fitness f*
```

WALKER

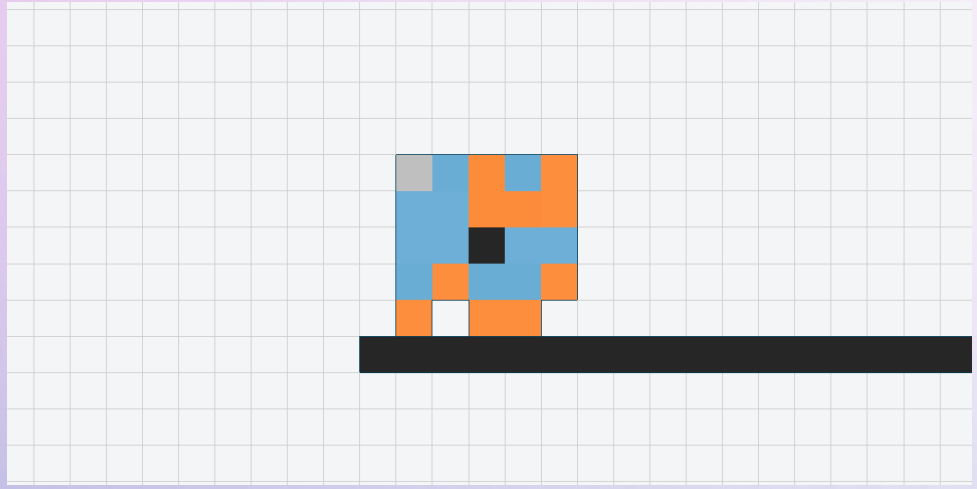
GEN #1
Population : 100
Generations : 20
Sub population : 5
Max Step : 100



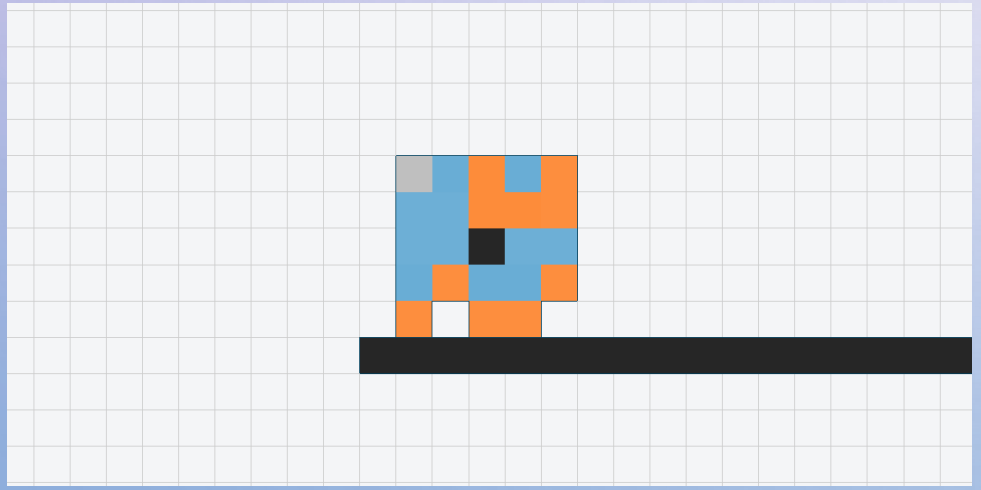
GEN #2
Population : 15
Generations : 50
Sub population : 10
Max Step : 200

GEN #3
Population : 5
Generations : 100
Sub population : 10
Max Step : 500

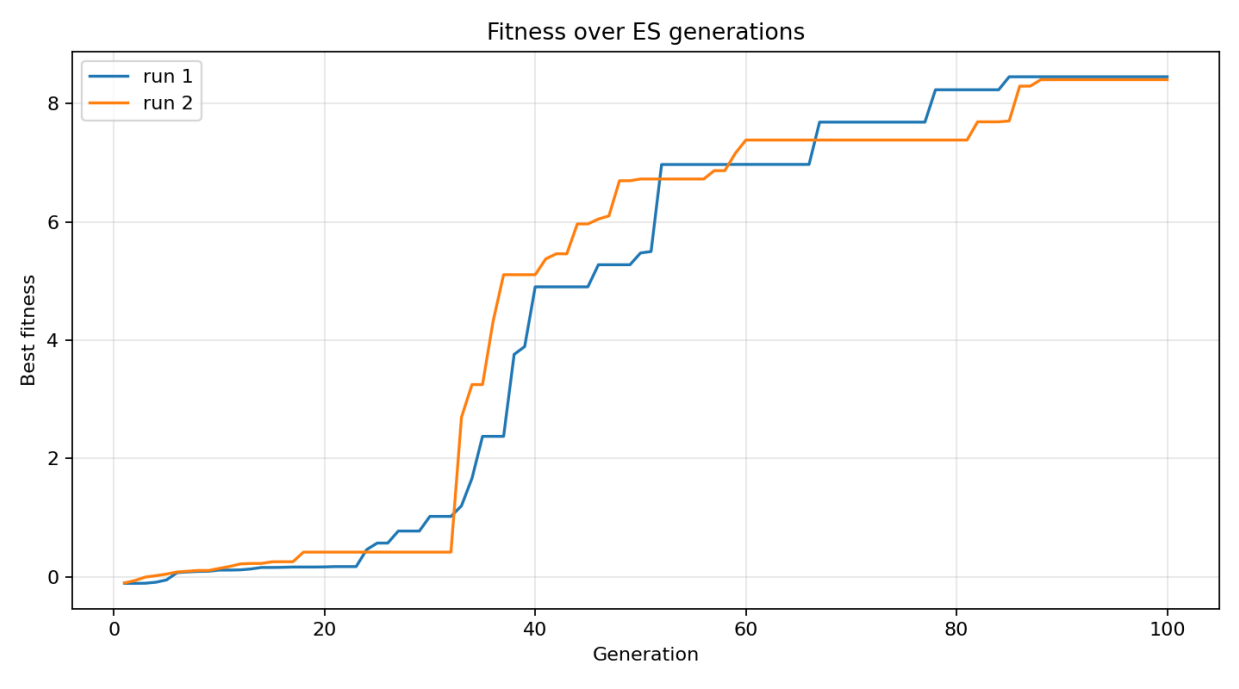
WALKER : 8.435



8.46



8.41



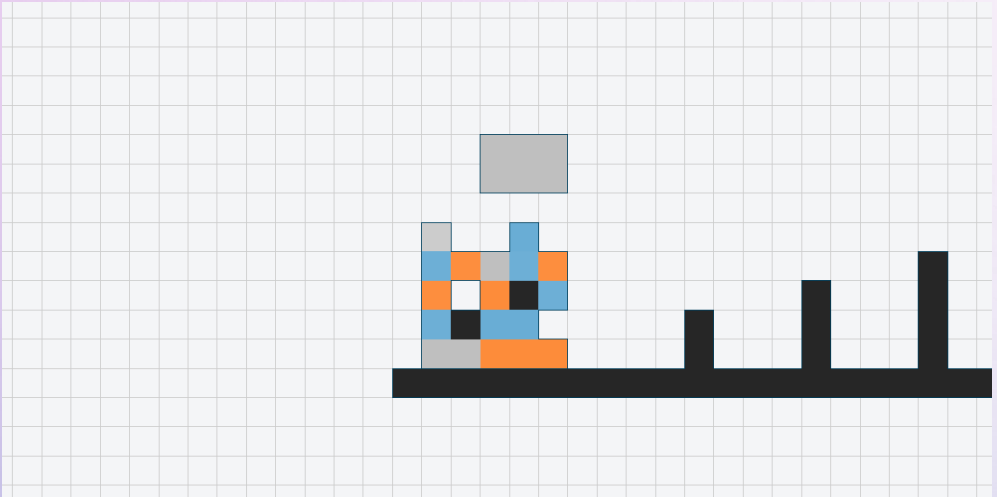
THROWER

GEN #1
Population : 100
Generations : 20
Sub population : 5
Max Step : 100

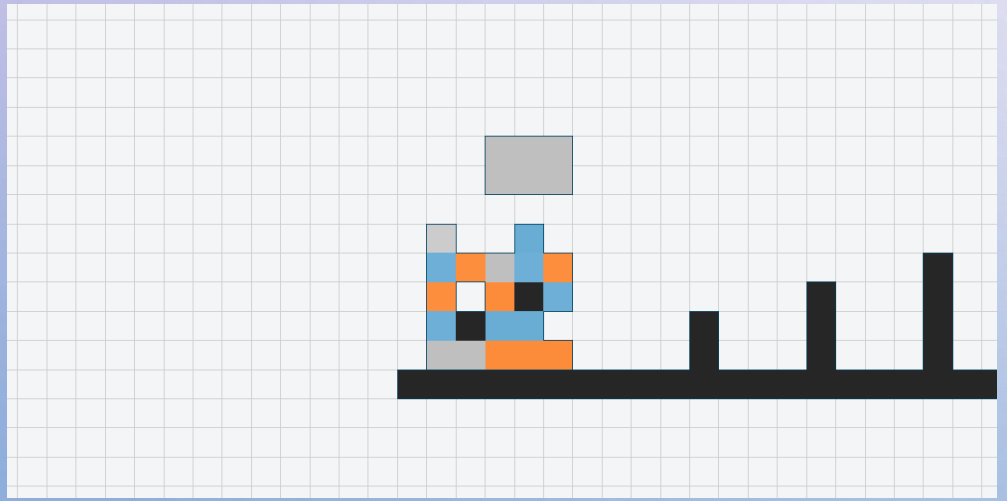
GEN #2
Population : 15
Generations : 50
Sub population : 10
Max Step : 200

GEN #3
Population : 5
Generations : 100
Sub population : 10
Max Step : 500

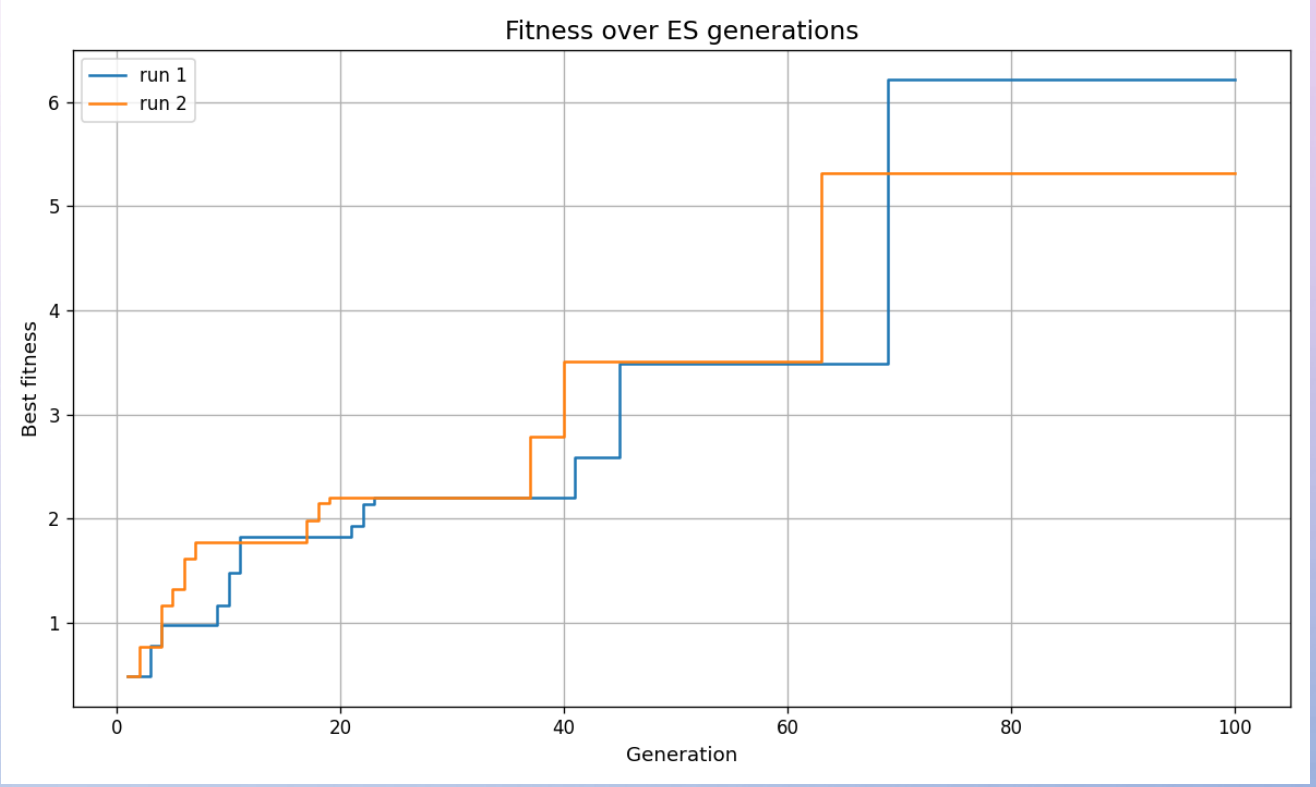
THROWER : 5.77



6.22



5.32



CLIMBER

Same code…Worse results…so some ideas

OPTIMIZING : BODY

Environment was not passing the current form…
But the general body shape was acceptable

```
env = make_env(env_name, robot=base)
```

We should look at optimizing the brain…

OPTIMIZING : BRAIN (PART I)

NN was returning the raw outputs (robot cells)…

```
def act(self, obs):
…
    return actions
```

Can be large, small, even negative…

```
def act(self, obs):
…
    return np.clip(actions, 0.6, 1.6)
```

Constraining the max expansion and contraction of each cell to be in a more realistic range

OPTIMIZING : BRAIN (PART II)

Random noise mutation was replacing the weights (genes)…

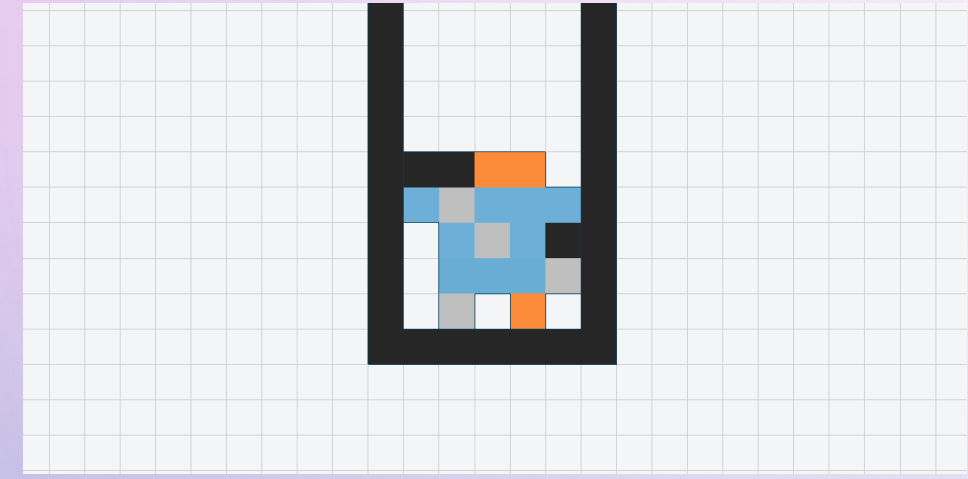
```
def mutate_ga(self):
…
    new_genes[~f] = noise
    return new_genes
```

Randomly replaces weights with noise

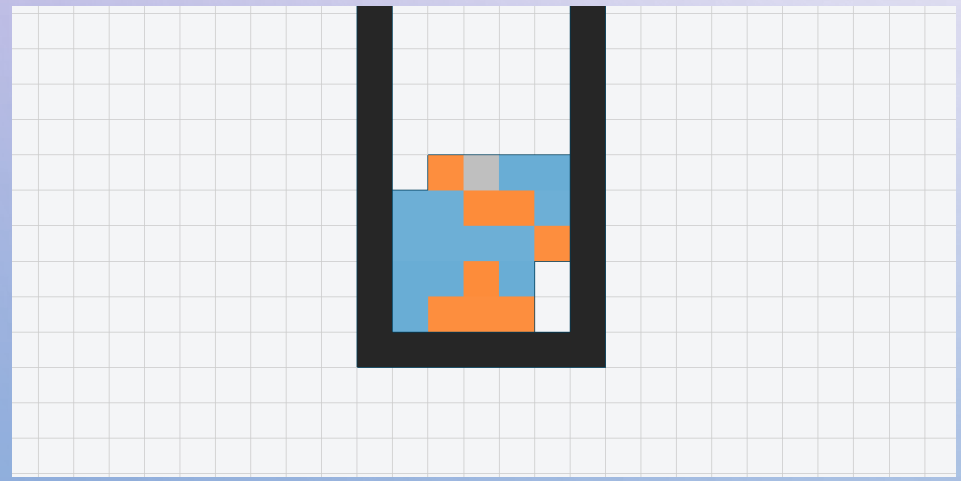
```
def mutate_ga(self, sigma=0.15):
…
    new_genes = genes + sigma*np.random.rand(genes.shape)
    return new_genes
```

Adding random gaussian noise to each weight

CLIMBER : 0.49

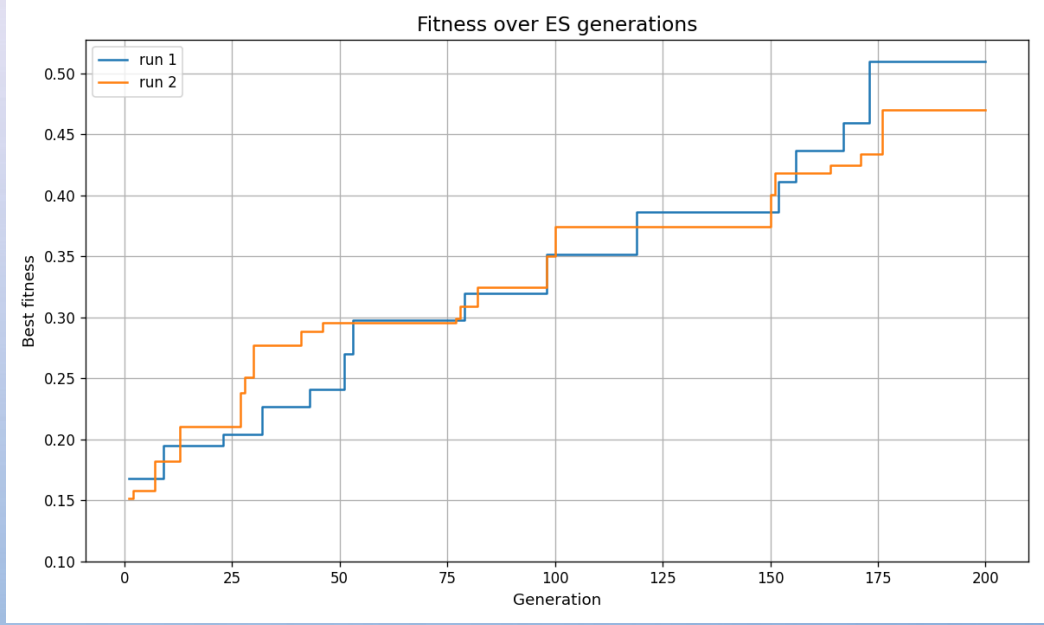


0.51



0.47

Another notable result

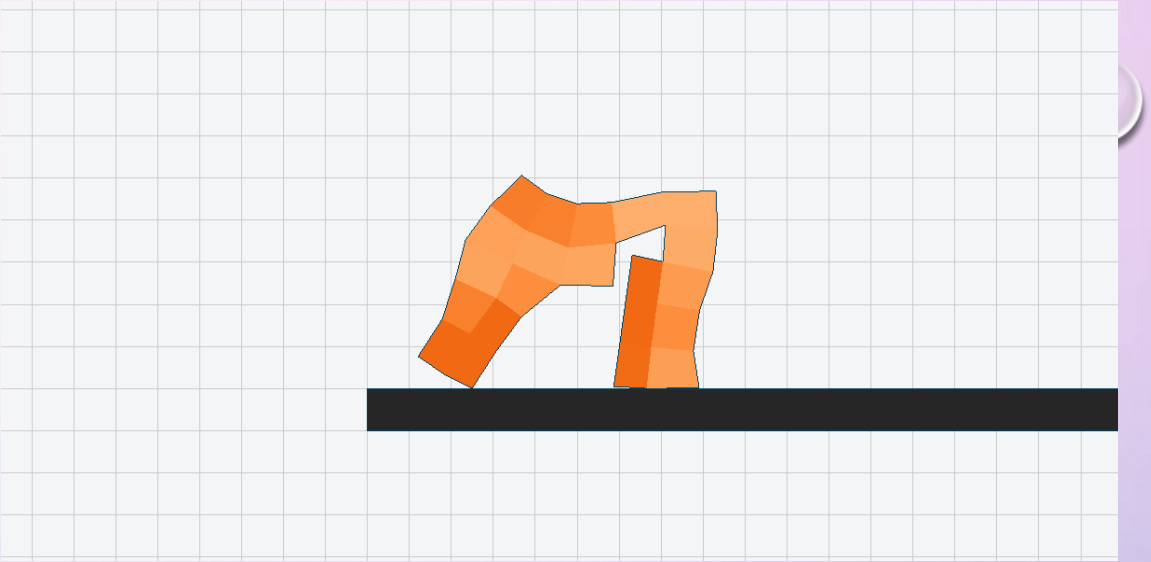
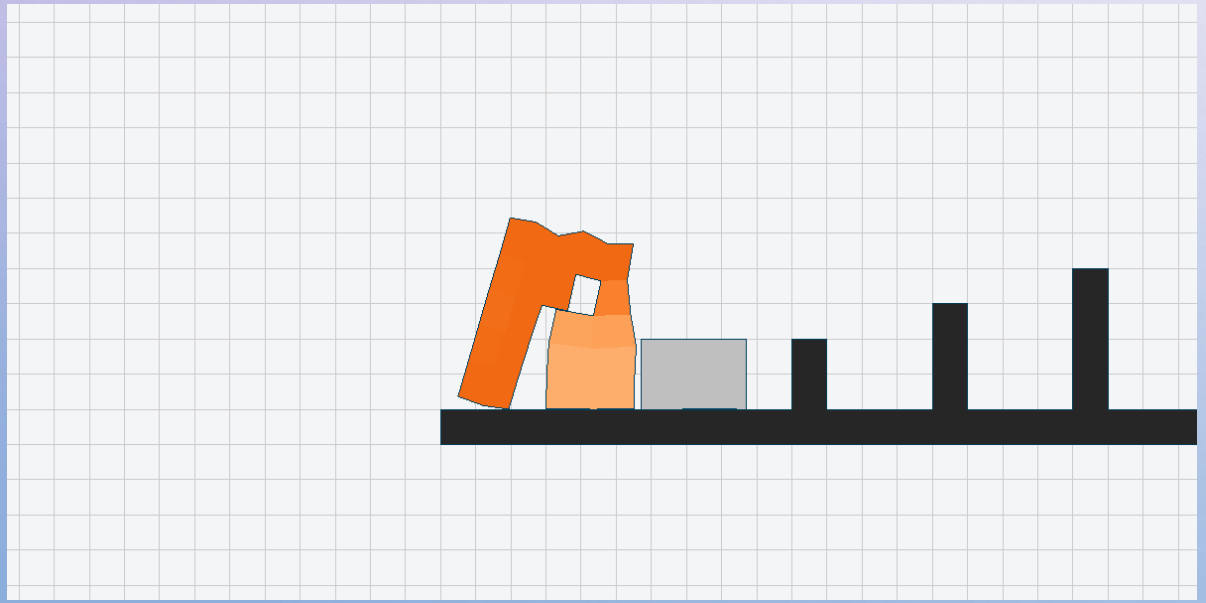
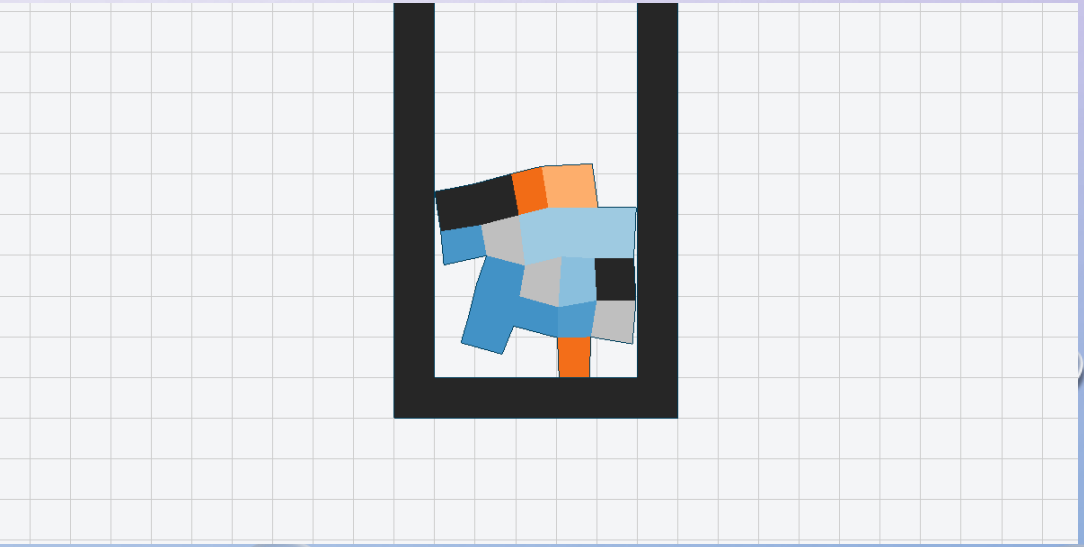


IMPROVING CLIMBER

WHY IT DOES NOT WORK :

- THE EVOLUTION WITH ONLY 1 MUTATION PER GENERATION CAN HARDLY LEARN THE MECHANICS TO CLIMB (GRIPPING…)
- IT CAN NOT LEARN CYCLES

IMPROVEMENT IDEAS:

- CHANGE THE CONTROLLER: INCREASE THE NN SIZE, CHANGE THE STRUCTURE…
- ADAPT THE FITNESS CALCULATION: REWARDS FOR GRIPPING TO THE SIDES, PENALTY FOR DROPPING TO THE GROUND…
- ADAPT THE GENERATION OPTIMIZATION ALGORITHM

THANK YOU